

Ayudantía 02: Filtro Notch

Power Electronic Control Fundamentals



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

Indice

1. Filtro Notch
2. Métodos de discretización
3. C-Script
4. Ejercicio Propuesto

Filtro Notch

$$F(s) = \frac{s^2 + \omega_0^2}{s^2 + \frac{\omega_0}{Q}s + \omega_0^2}$$

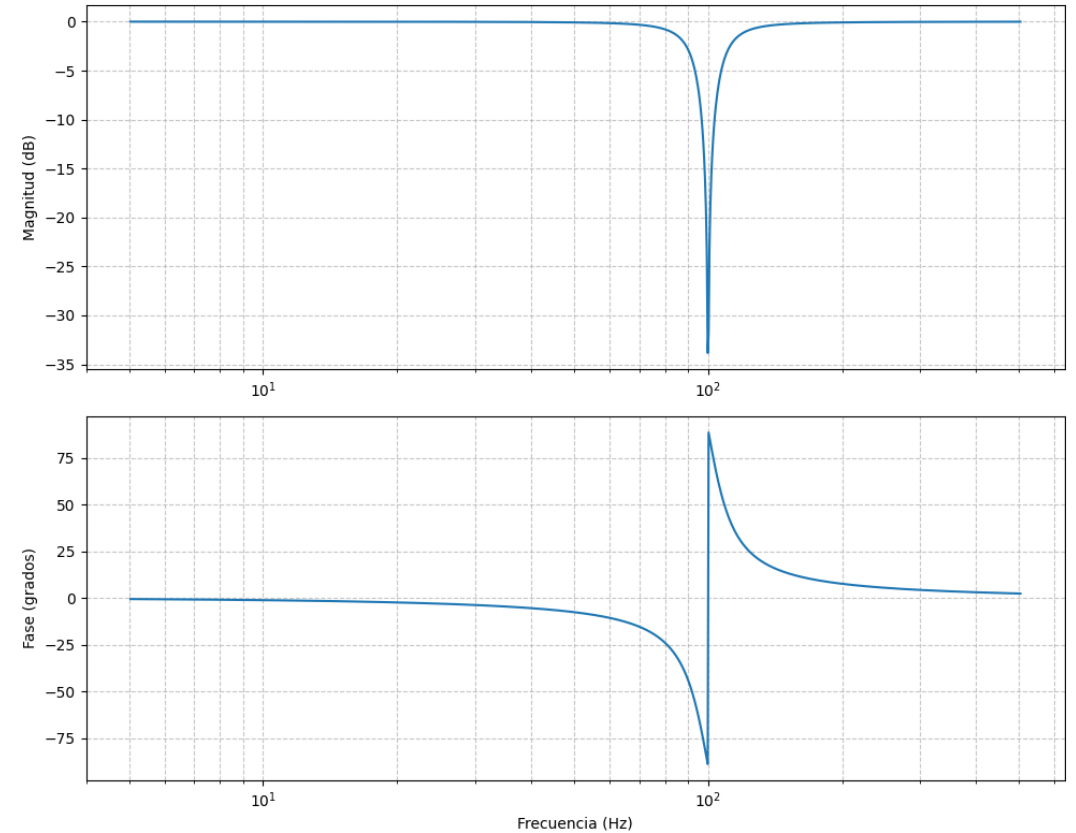


Diagrama de Bode de un Filtro Notch

Filtro Notch

El filtro notch es un tipo de filtro de rechazo de banda de la forma:

$$F(s) = \frac{s^2 + \omega_0^2}{s^2 + \frac{\omega_0}{Q}s + \omega_0^2}$$

que se ajusta para eliminar una frecuencia específica.

$$\omega_0 = 2\pi f_0$$

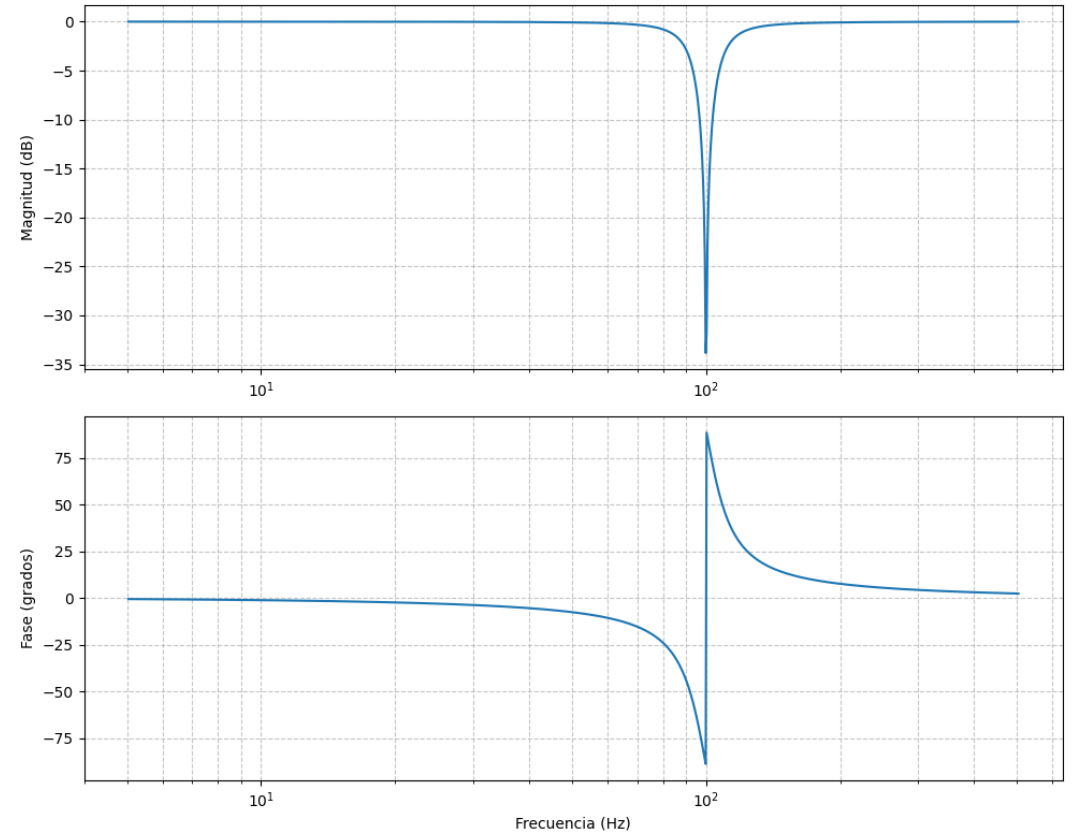


Diagrama de Bode de un Filtro Notch

Filtro Notch

$$F(s) = \frac{s^2 + \omega_0^2}{s^2 + \frac{\omega_0}{Q} + \omega_0^2}$$

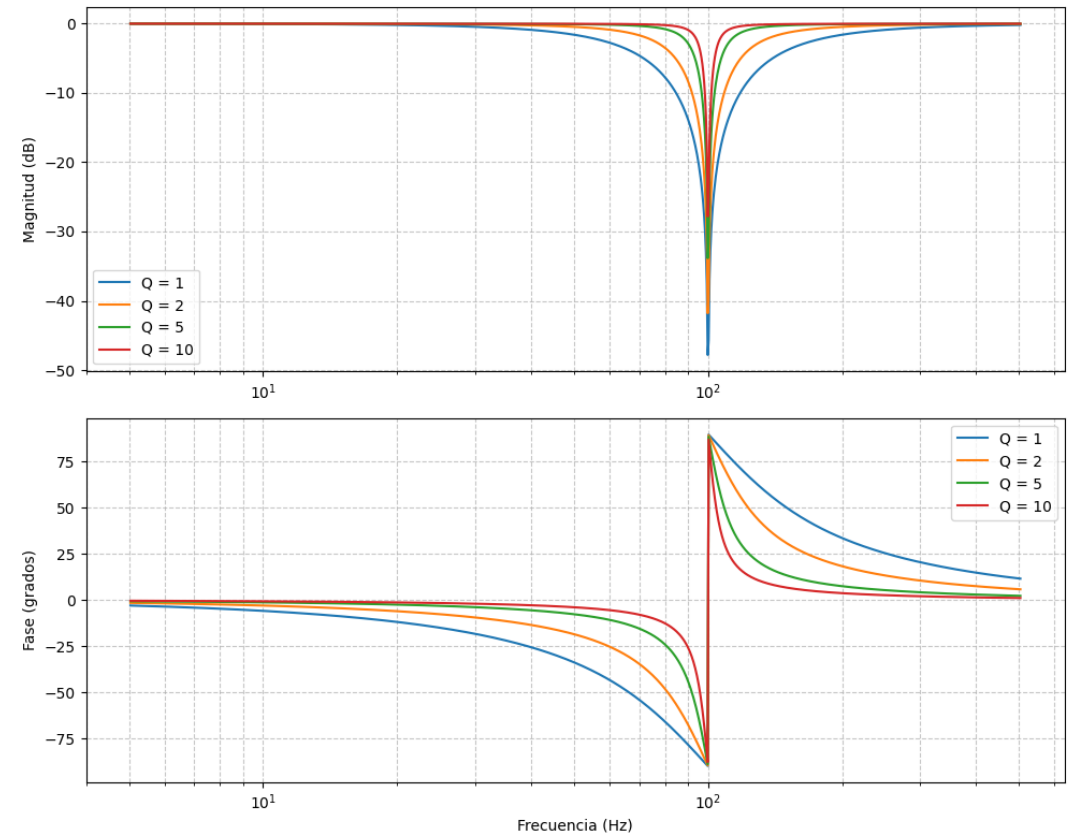


Diagrama de Bode de varios Filtros
Notch con distinto Q

Filtro Notch

Además de la frecuencia, el otro parámetro de diseño del filtro es el factor de calidad (Q).

$$F(s) = \frac{s^2 + \omega_0^2}{s^2 + \frac{\omega_0}{Q} + \omega_0^2}$$

Al seleccionar Q, existe un *trade-off* entre la banda de rechazo y el *settling time*.

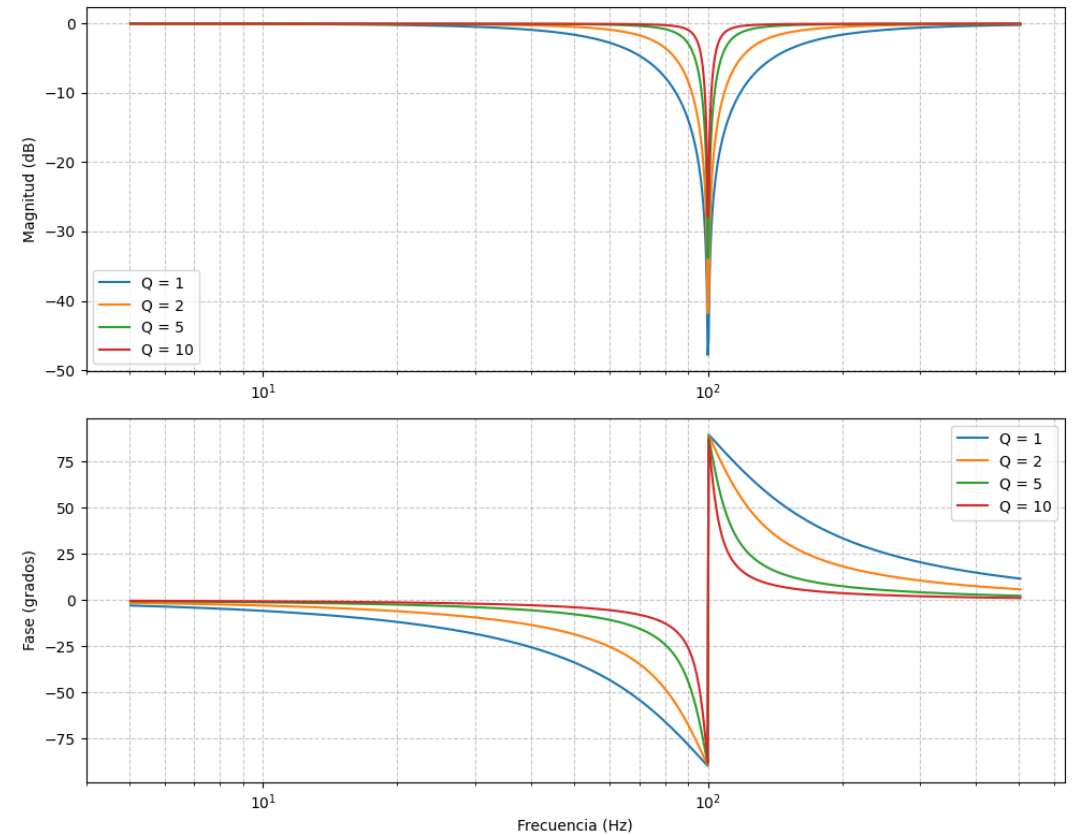


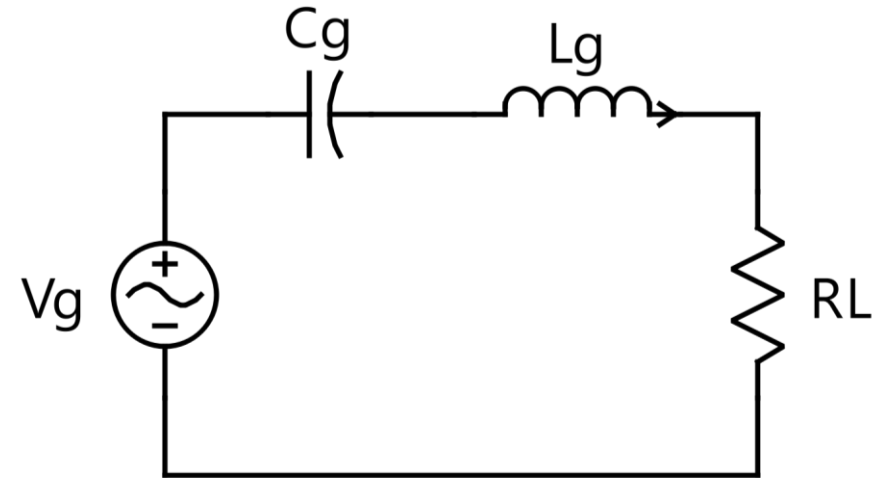
Diagrama de Bode de varios Filtros Notch con distinto Q

Caso de uso

$$v_g(t) = V_g \cos(\omega_g t)$$

$$i_g(t) = I_g \cos(\omega_g t - \phi)$$

$$p_g(t) = \frac{V_g I_g}{2} \cos(\phi) + \frac{V_g I_g}{2} \cos(2\omega_g t - \phi)$$



Entonces, la potencia de la fuente se compone de: potencia activa + potencia reactiva = señal DC + señal de frecuencia 2f.

Por lo tanto, podemos ajustar un filtro notch para eliminar la componente de frecuencia 2f, obteniendo como resultado la potencia activa.

Discretización

Cuando operamos en el dominio discreto (como al usar microcontroladores), no se puede representar de manera directa la naturaleza continua, por lo que es necesario utilizar aproximaciones tanto de la derivada como de la integral.

$$s \approx \frac{z - 1}{T_s}$$

Forward Euler

$$s \approx \frac{z - 1}{zT_s}$$

Backward Euler

$$s \approx \frac{2}{T_s} \frac{z - 1}{z + 1}$$

Bilinear (Tustin)

De esta manera, podemos obtener la función de transferencia en el espacio Z a partir de la función de transferencia F(s).

How C-Script Work

El bloque C-Script tiene (que nos interesen de momento) 5 bloques de código:

- *Code Declaration*: se utiliza para declaraciones y definiciones globales, además de incluir librerías.
- *Start Function Code*: es una función que se llamará al inicio de la simulación.
- *Output Function Code*: se llama cada vez que ocurre un cambio en la simulación.
- *Update Function Code*: se llama una vez al final de cada T_s y permite modelar comportamientos discretos.
- *Derivative Function Code*: permite modelar comportamiento continuo.

$$y = f_{output}(t, x_c, x_d, y)$$

$$x_d^+ = f_{update}(t, x_c, x_d, y)$$

$$\dot{x}_c = f_{derivative}(t, x_c, x_d, y)$$

How C-Script Work

$$\begin{aligned}y &= f_{output}(t, x_c, x_d, y) \\x_d^+ &= f_{update}(t, x_c, x_d, y) \\\dot{x}_c &= f_{derivative}(t, x_c, x_d, y)\end{aligned}$$

Para modelar un comportamiento discreto:

1. Definir en *Code Declaration* “DiscState(i)” para cada estado discreto.
2. Definir las actualizaciones en *Update Function*.
3. Definir la salida del bloque a partir de los estados en *Output Function*.

Para modelar un comportamiento continuo:

1. Definir en *Code Declaration* “ContState(i)” y “ContDeriv(i)” para cada estado continuo.
2. Definir el valor de la derivada de cada estado en *Derivative Function*.
3. Definir la salida del bloque a partir de los estados en *Output Function*.

Ejercicio propuesto

El ejercicio propuesto consiste en implementar un filtro notch, tanto continuo como discreto (usando Tustin), en el bloque C-Script.

$$F(s) = \frac{s^2 + \omega_0^2}{s^2 + \frac{\omega_0}{Q} s + \omega_0^2}$$

Función de transferencia del
filtro Notch

$$s \approx \frac{2}{T_s} \frac{z - 1}{z + 1}$$

Aproximación de Tustin

$$\begin{aligned} y &= f_{output}(t, x_c, x_d, y) \\ x_d^+ &= f_{update}(t, x_c, x_d, y) \\ \dot{x}_c &= f_{derivative}(t, x_c, x_d, y) \end{aligned}$$

Funciones del C-Script